BUITEMS

Faculty of Information Communication Technology
Department of Data Science

Lab Report

Computer Networks (CS 305)

Packet Sniffer and Network Traffic Analyzer using Python

Instructor	Ms. Poma Panezai
Course Code	CS 305
Semester	3rd Semester — Spring 2026
Group Members	1. Manahil Nousherwnii 2. Farhaan Sher
Submission Date	13 July 2026

1. Introduction

A packet sniffer is a software tool that captures and analyzes data packets traveling across a network interface. Every time a user opens a website, sends a message, or downloads a file, packets are transmitted across the network following specific protocols. Packet sniffing enables network administrators, security analysts, and developers to observe this real communication and understand how different protocols operate in practice.

This project involves the development of a Python-based Packet Sniffer and Network Traffic Analyzer. The tool captures live network packets (or reads from a saved .pcap file), extracts useful information from each packet, and presents a structured traffic summary. The project provides hands-on experience with key Computer Networks concepts including IP addresses, MAC addresses, ports, and protocols such as TCP, UDP, ICMP, DNS, HTTP, and HTTPS.

Network traffic analysis is important for:

- Diagnosing and troubleshooting network problems
- Identifying unusual or suspicious activity on a network
- Understanding bandwidth usage and protocol distribution
- Verifying the correct operation of network applications

2. Objectives

The main objectives of this project are:

- To understand how network packets are transmitted across a network
- To capture and analyze network packets using Python
- To identify basic protocol types such as TCP, UDP, ICMP, DNS, HTTP, and HTTPS
- To extract useful packet information including source IP, destination IP, source port, destination port, protocol type, and packet size
- To count and summarize captured packets by protocol
- To generate simple traffic statistics and visualizations (graphs and charts)
- To save captured packet data into a structured CSV file
- To prepare a project report and present the working tool

3. Tools and Technologies

The following tools, technologies, and Python libraries were used in this project:

Tool / Library	Purpose
Python 3.8+	Primary programming language for the entire project
Scapy	Packet capturing and packet-level analysis
PyShark	Alternative packet analysis using Wireshark's dissectors
Pandas	Storing and processing captured packet data

Matplotlib	Creating protocol distribution graphs and charts
CSV module	Saving packet records to a CSV file
Datetime module	Recording packet capture timestamps
Wireshark	Verifying and cross-checking captured packets
GitHub	Version control and repository hosting
Jupyter Notebook / VS Code	Development and testing environment

4. Methodology

The project was completed in a structured series of steps:

Step 1 — Study Basic Network Concepts

Before writing any code, we reviewed fundamental networking concepts including IP addressing, MAC addressing, port numbers, and the key protocols: TCP, UDP, ICMP, DNS, HTTP, and HTTPS. We also studied how packets are structured at different OSI model layers.

Step 2 — Install Required Tools

We installed Python 3, Wireshark, and the required Python libraries (Scapy, Pandas, Matplotlib). We verified that live packet capturing was functional on our system by running a basic Scapy sniff test (administrator/root permissions were required).

Step 3 — Capture Network Packets

The program was configured to capture packets from the active network interface. We tested both live packet capturing using Scapy's sniff() function and reading packets from a saved .pcap file using PyShark for verification.

Step 4 — Extract Packet Information

For each captured packet, the following fields were extracted:

- Time of capture
- Source IP address
- Destination IP address
- Source port (where available)
- Destination port (where available)
- Protocol type (TCP, UDP, ICMP, etc.)
- Packet length (bytes)
- **Step 5 — Store Packet Data**

Extracted packet information was stored in a structured Pandas DataFrame and then exported to a CSV file (captured_packets.csv) for further analysis and reference.

Step 6 — Analyze the Traffic

The program analyzed captured packets and generated a summary report including: total packet count, count by protocol (TCP/UDP/ICMP/Other), top source IP addresses, top destination IP addresses, and most commonly used destination ports.

Step 7 — Visualize the Results

Using Matplotlib, we created visual graphs including a protocol distribution bar chart, a protocol distribution pie chart, and a top source IP addresses graph to communicate traffic patterns clearly.

5. Implementation

5.1 Main Code Modules

The project was organized into three main Python modules:

- packet_sniffer.py — Handles live packet capture and .pcap file reading using Scapy
- analyzer.py — Processes the captured data, generates statistics and summaries
- visualizer.py — Creates graphs and charts using Matplotlib

5.2 Packet Capturing Method

Live packets were captured using Scapy's sniff() function with a configurable packet count limit. Each packet was parsed for its IP layer, transport layer (TCP/UDP), and ICMP layer where applicable. The packet capture required administrator privileges to access the raw network interface.

5.3 Packet Analysis Method

Each captured packet was inspected for the presence of IP, TCP, UDP, and ICMP layers. The protocol was identified and the relevant fields (source/destination IP, ports, length) were extracted and stored in a dictionary. These dictionaries were collected into a list and converted to a Pandas DataFrame for efficient processing.

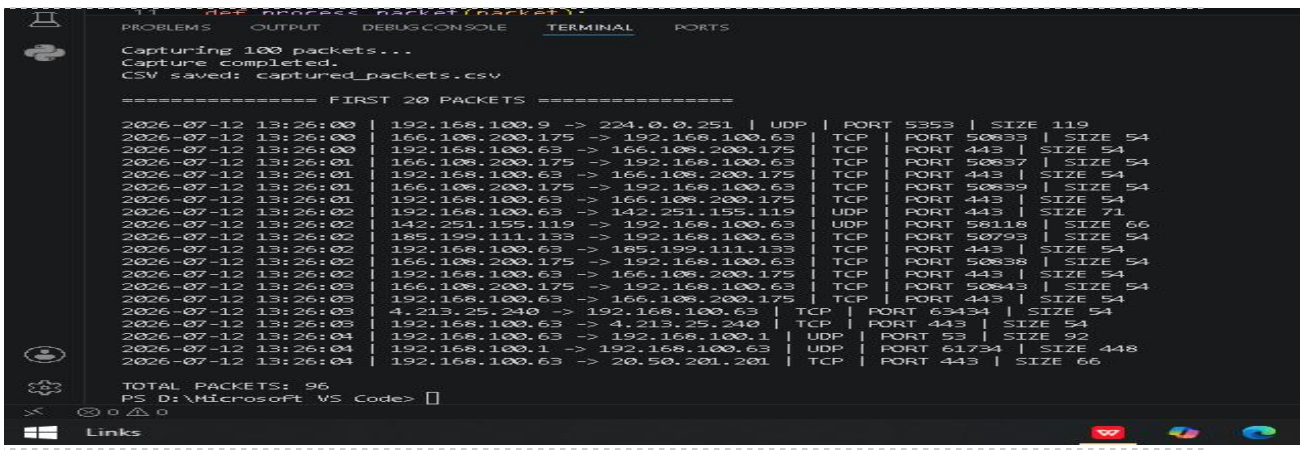
5.4 Data Storage Method

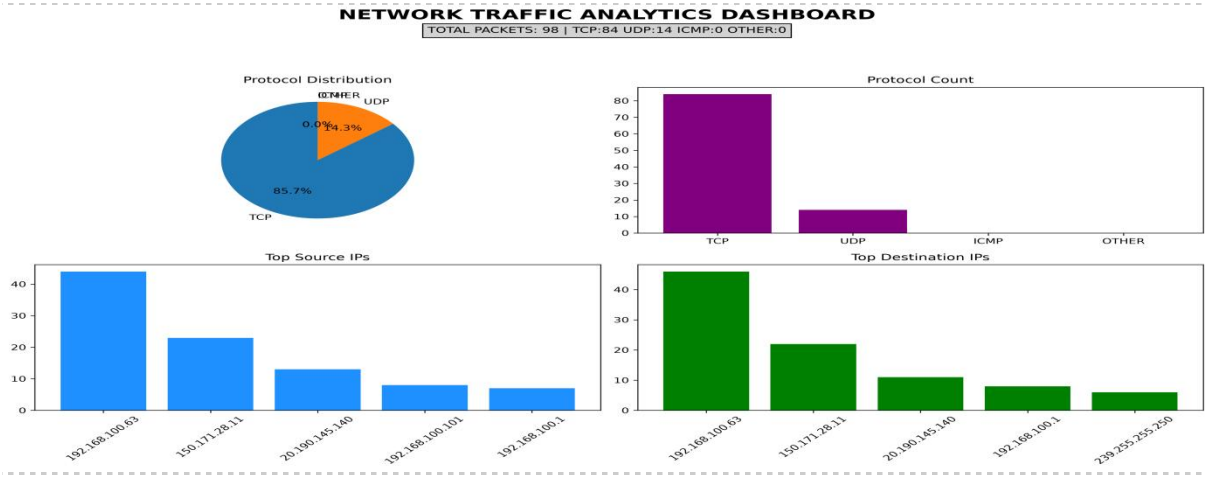
The DataFrame was saved to a CSV file using Pandas' to_csv() method. This allowed the captured packet data to be opened in spreadsheet applications for manual review and reloaded in future analysis sessions.

6. Results and Screenshots

The following outputs were produced by the working tool:

- A real-time list of captured packets displayed in the terminal with source IP, destination IP, protocol, and packet size
- A CSV file (captured_packets.csv) containing all captured packet records
- A protocol distribution bar chart showing the count of TCP, UDP, ICMP, and other packets
- A protocol distribution pie chart showing the percentage share of each protocol
- A summary of the top 5 source IP addresses by packet count
- A summary of the most commonly used destination ports





7. Conclusion

This project provided a practical and hands-on understanding of how network packets travel across a network and how different protocols operate at the transport and network layers. By building a Python-based Packet Sniffer and Network Traffic Analyzer, we were able to observe real network communication rather than only studying theoretical concepts.

Through this project, we learned how to use Scapy for live packet capturing, how to extract and interpret key fields from network packets, how to store and process structured data using Pandas, and how to generate meaningful visual summaries of network traffic using Matplotlib. We also gained experience with organizing a technical project using GitHub.

The project directly connected our coursework in Computer Networks to a real-world tool used in network administration and cybersecurity. Understanding packet sniffing is a foundational skill for diagnosing network problems, monitoring traffic behavior, and identifying potential security threats

8. References

- Scapy Official Documentation — https://scapy.readthedocs.io
- Wireshark User Guide — https://www.wireshark.org/docs/wsug_html/
- Pandas Documentation — https://pandas.pydata.org/docs/
- Matplotlib Documentation — https://matplotlib.org/stable/contents.html
- Forouzan, B. A. — Data Communications and Networking, 5th Edition, McGraw-Hill
- Kurose, J. F. & Ross, K. W. — Computer Networking: A Top-Down Approach, 8th Edition

9.Github repositories

https://github.com/tareenfarhaan-blip/packet-sniffer-network-analyzer